

Stream

Interfacce Funzionali

Queste sono le fondamenta. Ogni interfaccia rappresenta un "comportamento" che puoi passare come parametro.

- `Predicate<T>` (Il Filtro)
- **Cosa fa:** Prende un input `T` e restituisce un `boolean`.
- *Esempio Pratico:*

```
1 Predicate<Integer> isPari = x -> x % 2 == 0;  
2 isPari.test(4); // Restituisce true
```

- `Function<T, R>` (Il Trasformatore)
- **Cosa fa:** Prende un input `T` e restituisce un output `R`.
- *Esempio Pratico:*

```
1 Function<String, Integer> conta = str -> str.length();  
2 conta.apply("Java"); // Restituisce 4
```

- `Consumer<T>` (Il Consumatore)
- **Cosa fa:** Prende un input `T` e non restituisce nulla (`void`). Modifica o stampa.
- *Esempio Pratico:*

```
1 Consumer<String> stampa = msg -> System.out.println(msg);  
2 stampa.accept("Ciao"); // Stampa "Ciao" in console
```

- `Supplier<T>` (Il Fabbrikante)
- **Cosa fa:** Non prende input (`()`), ma restituisce un output `T`.
- *Esempio Pratico:*

```
1 Supplier<Double> random = () -> Math.random();  
2 random.get(); // Restituisce un numero casuale (es. 0.83)
```

- `UnaryOperator<T>` & `BinaryOperator<T>` (I Calcolatori)
- **Cosa fanno:** Funzioni speciali dove input e output sono dello stesso tipo. `Unary` prende un parametro, `Binary` ne prende due.
- *Esempio Unary:*

```
1 UnaryOperator<Integer> raddoppia = n -> n * 2;  
2 raddoppia.apply(5); // Restituisce 10
```

- *Esempio Binary:*

```
1 BinaryOperator<Integer> somma = (a, b) -> a + b;  
2 somma.apply(3, 4); // Restituisce 7
```

Method References (::)

Una sintassi più corta per le lambda quando richiami semplicemente un metodo già esistente:

- `x -> System.out.println(x)` → `System.out::println`
- `str -> str.toUpperCase()` → `String::toUpperCase`
- `() -> new ArrayList<>()` → `ArrayList::new`

La Stream API: Regole d'Oro

Uno Stream **NON** è una struttura dati, ma un "tubo" in cui i dati scorrono.

1. **Non muta la sorgente:** I dati originali non cambiano mai.
2. **Lazy (Pigro):** Le operazioni intermedie non vengono eseguite finché non invochi un'operazione terminale.
3. **Trappola da Esame:** Uno stream si *consuma*. Una volta chiamata un'operazione terminale, lo stream è chiuso. Riutilizzarlo lancia `IllegalStateException`.

Creazione (L'origine del Flusso)

- **Da Collection:**

```
1 List<String> nomi = List.of("Anna", "Bob");
```

```
2 Stream<String> s = nomi.stream();
```

- **Da Array (Nativi):**

```
1 int[] numeri = {1, 2, 3};
2 IntStream s = Arrays.stream(numeri);
```

- **Da Valori Diretti:**

```
1 Stream<Integer> s = Stream.of(10, 20, 30);
```

- **Generazione Infinita:**

```
1 Stream<Double> s = Stream.generate(Math::random);
```

Operazioni Intermedie

Restituiscono sempre un nuovo Stream. Sono lazy.

- `filter(Predicate)` : Fa passare solo gli elementi che rispettano la condizione.

```
1 stream.filter(n -> n > 10) // Tiene solo i maggiori di 10
```

- `map(Function)` : Trasforma ogni elemento (es. estrae un campo).

```
1 stream.map(Persona::getName) // Da Stream<Persona> a Stream<String>
2
3 List<String> nomi = Arrays.asList("Mario", "Luigi", "Peach");
4
5 // Vogliamo sapere la lunghezza di ogni nome
6 List<Integer> lunghezze = nomi.stream()
7     .map(nome -> nome.length()) // Mappa: da String passiamo a Integer
8     .collect(Collectors.toList());
9
10 // Input (3 elementi): ["Mario", "Luigi", "Peach"]
11 // Output (3 elementi): [5, 5, 5]
```

- `flatMap(Function)` : Appiattisce stream annidati ("schiaccia" liste dentro liste).

```
1 streamDiListe.flatMap(lista -> lista.stream()) // Unica lista piatta
2
3 // Abbiamo una lista di classi, ogni classe è una lista di nomi
4 List<List<String>> scuola = Arrays.asList(
5     Arrays.asList("Alice", "Bob"), // Classe 1
6     Arrays.asList("Carlo", "Diana") // Classe 2
7 );
8
9 // Ottieni uno Stream<String> (Solo i nomi sfusi sul nastro)
10 List<String> tuttiGliStudenti = scuola.stream()
11     .flatMap(classe -> classe.stream()) // Apre il cestino e ne fa scorrere il contenuto
12     .collect(Collectors.toList());
13
14 // Output: ["Alice", "Bob", "Carlo", "Diana"]
```

- `mapToObj(function)` crea stream di oggetti compessi

```
1 // Vogliamo creare una lista di stringhe numerate
2 List<String> etichette = IntStream.rangeClosed(1, 3) // Partiamo da primitivi: 1, 2, 3
3     .mapToObj(n -> "Posto numero: " + n) // Promuoviamo ogni 'int' a 'String' (Oggetto)
4     .collect(Collectors.toList()); // Ora possiamo usare i Collectors!
5
6 // Output: ["Posto numero: 1", "Posto numero: 2", "Posto numero: 3"]
```

- `distinct()` : Rimuove i duplicati (basandosi su `equals()`).

```
1 Stream.of(1, 2, 2, 3).distinct() // Diventa: 1, 2, 3
```

- `sorted()` / ``sorted(Comparator)``: Ordina gli elementi.

```
1 stream.sorted(Comparator.reverseOrder()) // Ordine decrescente
```

- `limit(n)` : Tronca lo stream ai primi `n` elementi.

```
1 stream.limit(5) // Prende solo i primi 5
```

- `skip(n)` : Ignota i primi `n` elementi.

```
1 stream.skip(2) // Parte dal 3° elemento in poi
```

- `peek(Consumer)` : "Spia" l'elemento per debug, senza modificarlo.

```
1 stream.peek(x -> System.out.println("Transita: " + x))
```

- `boxed()` : inscatola i tipi primitivi in un wrapper

```
1 List<Integer> listaNumeri = IntStream.range(1, 5).boxed() // Da IntStream a Stream<Integer>
2 .collect(Collectors.toList());
```

- `takeWhile(Predicate)` `dropWhile(Predicate)` `takeWhile` fa passare gli elementi *finché* la condizione è vera, al primo 'false' chiude il flusso. `dropWhile` scarta gli elementi *finché* la condizione è vera, al primo 'false' fa passare tutto il resto. comodi su stream ordinati.

```
1 List<Integer> presi = Stream.of(1, 2, 3, 4, 5, 9)
2   .takeWhile(n -> n < 4)
3   .collect(Collectors.toList());
4 // Output: [1, 2, 3] (si ferma al 4, ignora il resto)
```

- `Stream.concat(Stream, Stream)` unisce due flussi

```
1 Stream<String> s1 = Stream.of("A", "B");
2 Stream<String> s2 = Stream.of("C", "D");
3 Stream<String> uniti = Stream.concat(s1, s2);
4 // Flusso risultante: "A", "B", "C", "D"
```

Operazioni Terminali (La Chiusura)

Chiudono lo stream e restituiscono un valore concreto, un `Optional`, o `void`.

- `collect(Collector)` : Imbottiglia i risultati finali in List, Set o Map.

```
1 List<String> ris = stream.collect(Collectors.toList());
```

- `reduce(BinaryOperator)` : Riduce tutti i dati a un singolo valore cumulativo.

```
1 Optional<Integer> somma = stream.reduce((parziale, prossimo) -> parziale + prossimo);
2
3 // con valore di partenza
4 Integer somma = stream.reduce(0, (parziale, prossimo) -> parziale + prossimo);
5
6 //trova la parola piu lunga
7 .reduce("", (a, b) -> a.length() > b.length() ? a : b);
```

- `forEach(Consumer)` : Esegue un'azione terminale su ogni elemento.

```
1 stream.forEach(System.out::println);
```

- `count()` : Conta quanti elementi sono arrivati alla fine.

```
1 long quanti = stream.count();
```

- `min(Comparator)` / `max(Comparator)` : Trova l'estremo.

```
1 Optional<Integer> max = stream.max(Integer::compareTo);
```

- `findFirst()` : Prende il primo elemento utile.

```
1 Optional<String> primo = stream.filter(s -> s.startsWith("A")).findFirst();
```

- I Matchers (Restituiscono `boolean`):

```
1 boolean cEUnPari = stream.anyMatch(n -> n % 2 == 0); // Almeno uno
```

```

2 boolean tuttiPositivi = stream.allMatch(n -> n > 0); // Tutti quanti
3 boolean nessunoZero = stream.noneMatch(n -> n == 0); // Nessuno

```

- `Collectors.joining()` unisce più stringhe in una sola

```

1 List<String> nomi = Arrays.asList("Anna", "Marco", "Luca");
2
3 // 1. Unione semplice (tutto attaccato)
4 String uniti = nomi.stream().collect(Collectors.joining());
5 // Output: "AnnaMarcoLuca"
6
7 // 2. Unione con delimitatore (IL PIÙ USATO ALL'ESAME)
8 String conVirgola = nomi.stream().collect(Collectors.joining(", "));
9 // Output: "Anna, Marco, Luca"
10
11 // 3. Unione con delimitatore, prefisso e suffisso
12 String completo = nomi.stream().collect(Collectors.joining(", ", "[", "]"));
13 // Output: "[Anna, Marco, Luca]"

```

- `Collectors.partitioningBy()` divide il flusso in **esattamente due fazioni** (Vero o Falso) basandosi su un `Predicate`.

```

1 List<Integer> numeri = Arrays.asList(1, 2, 3, 4, 5, 6);
2
3 // Vogliamo dividere i numeri in "Pari" (true) e "Dispari" (false)
4 Map<Boolean, List<Integer>> partizione = numeri.stream()
5     .collect(Collectors.partitioningBy(n -> n % 2 == 0));
6
7 System.out.println("Pari: " + partizione.get(true)); // Output: Pari: [2, 4, 6]
8 System.out.println("Dispari: " + partizione.get(false)); // Output: Dispari: [1, 3, 5]

```

- `Collectors.groupingBy()` Prende gli elementi del tuo stream e li smista all'interno di una `Map`.

```

1 List<String> animali = Arrays.asList("Cane", "Gatto", "Lupo", "Oca", "Panda");
2
3 // Vogliamo raggruppare gli animali in base alla lunghezza del loro nome
4 Map<Integer, List<String>> raggruppatiPerLunghezza = animali.stream()
5     .collect(Collectors.groupingBy(String::length));
6
7 // Risultato nella Mappa:
8 // Chiave 3 -> ["Oca"]
9 // Chiave 4 -> ["Cane", "Lupo"]
10 // Chiave 5 -> ["Gatto", "Panda"]
11
12 // Contiamo quanti animali ci sono per ogni lunghezza
13 Map<Integer, Long> conteggioPerLunghezza = animali.stream()
14     .collect(Collectors.groupingBy(
15         String::length, // 1. Criterio di raggruppamento (La Chiave)
16         Collectors.counting() // 2. Cosa fare con gli elementi nel secchiello (Il Valore)
17     ));
18
19 // Risultato:
20 // 3 -> 1 (perché c'è solo "Oca")
21 // 4 -> 2 (perché ci sono "Cane" e "Lupo")
22 // 5 -> 2 (perché ci sono "Gatto" e "Panda")

```

- `Collectors.toMap()` posiziona i valori in una mappa

```

1 st.collect(Collectors.toMap(keyMapper, valueMapper));

```

- `toArray(IntFunction)` converte lo stream in un array tipizzato

```

1 String[] arrayNomi = streamNomi.toArray(String[]::new); //chiama il cotruttore
2 // Se lo stream è di primitivi (es. IntStream), basta solo .toArray()
3 int[] arrayNumeri = intStream.toArray();

```

- `Collectors.mapping` estrarre o trasformare un dato *mentre* lo si sta raccogliendo dentro un `groupingBy`

```

1 Map<String, List<String>> nomiPerCorso = studenti.stream()
2     .collect(Collectors.groupingBy(
3         Studente::getCorso,
4         Collectors.mapping(Studente::getNome, Collectors.toList()) // Estrae il nome e lo mette in lista
5     ));

```

- `Collectors.collectingAndThen()` accogli e fai un'ultima modifica

```
1 List<String> listaImmutabile = stream.collect(
2     Collectors.collectingAndThen(
3         Collectors.toList(),           // 1. Prima crea la lista
4         Collections::unmodifiableList // 2. Poi la rende immutabile
5     )
6 );
```

- `Collectors.minBy(Comparator)` `Collectors.maxBy(Comparator)` Trova l'elemento minimo o massimo durante una fase di raccolta.

```
1 Map<String, Optional<Studente>> migliorePerCorso = studenti.stream()
2     .collect(Collectors.groupingBy(
3         Studente::getCorso,
4         Collectors.maxBy(Comparator.comparing(Studente::getVoto))
5     ));
```

Stream Primitivi (Prestazioni ottimali)

Per evitare l'autoboxing (es. `Integer` vs `int`), usa `IntStream`, `DoubleStream`, `LongStream`. Si ottengono con metodi come `mapToInt()`.

- `sum()` : Calcola la somma totale.

```
1 int totale = intStream.sum();
```

- `average()` : Calcola la media aritmetica.

```
1 OptionalDouble media = intStream.average();
```

- `summaryStatistics()` : Il coltellino svizzero (Count, Min, Max, Sum, Avg).

```
1 IntSummaryStatistics stats = intStream.summaryStatistics();
2 System.out.println(stats.getMax());
```

- `range()` sostituisce il ciclo for

```
1 IntStream.range(0, n) // 1. Crea uno stream di indici: da 0 a (n - 1)
```

- `Stream.iterate()` sostituisce un for complesso

```
1 // Vogliamo generare le prime 5 potenze di 2
2 List<Integer> potenze = Stream.iterate(2, n -> n * 2) // Parto da 2, e ogni volta raddoppio
3     .limit(5) // FONDAMENTALE: mi fermo dopo 5 giri
4     .collect(Collectors.toList());
```

Optional<T> (Come salvarsi dalle NullPointerException)

Gli stream restituiscono un `Optional` quando il risultato potrebbe non esistere (es. cerchi un elemento ma lo stream è vuoto).

- `isPresent()` : Controlla se la scatola è piena.

```
1 if (opt.isPresent()) { ... }
```

- `ifPresent(Consumer)` : Se c'è qualcosa, esegui il codice.

```
1 opt.ifPresent(val -> System.out.println("Trovato: " + val));
```

- `get()` : Estrae brutalmente il valore (Lancia eccezione se vuoto! Attenzione all'esame).

```
1 String valore = opt.get();
```

- `orElse(Default)` : Ottimo piano B. Se vuoto, usa il valore indicato.

```
1 String finale = opt.orElse("Valore di Default");
```

- `orElseThrow()` : Lancia esplicitamente un'eccezione custom se vuoto.

```
1 String sicuro = opt.orElseThrow(() -> new RuntimeException("Vuoto!"));
```

Snippet Strategici

A. Il Classico: Filtra, Mappa, Ordina e Colleziona

```
1 List<String> parole = Arrays.asList("Albero", "casa", "Amico", "cane", "Ape");
2 List<String> risultato = parole.stream()
3     .filter(p -> p.toLowerCase().startsWith("a")) // 1. Filtra
4     .map(String::toUpperCase)                    // 2. Mappa (Method Reference)
5     .sorted()                                    // 3. Ordina
6     .collect(Collectors.toList());               // 4. Termina e raccoglie
7 // Output: [ALBERO, AMICO, APE]
```

B. Estrazione Campo (Oggetto → Campo specifico)

```
1 Set<String> nomiBravi = studenti.stream()
2     .filter(s -> s.getVoto() > 25)
3     .map(Studente::getNome) // Estrae solo la Stringa 'nome'
4     .collect(Collectors.toSet()); // toSet() elimina automaticamente gli omonimi
```

C. Il Potere di `groupingBy` (Creare Mappe)

```
1 List<String> parole = Arrays.asList("java", "c", "go", "php");
2 Map<Integer, List<String>> parolePerLunghezza = parole.stream()
3     .collect(Collectors.groupingBy(String::length));
4 // Chiave: lunghezza (es. 4) -> Valore: Lista di parole (es. ["java"])
```

D. Somma ottimizzata (`mapToInt`)

```
1 int sommaVoti = studenti.stream()
2     .mapToInt(Studente::getVoto) // Genera un IntStream primitivo
3     .sum();                       // Operazione terminale super efficiente
```

E. Regola d'oro delle Lambda: Variabili "Effectively Final"

Le Lambda non possono modificare le variabili dichiarate al di fuori di esse.

```
1 // ERRORE DA BOCCIATURA:
2 int contatore = 0;
3 lista.stream().forEach(x -> contatore++); // Errore di compilazione!
4 // SOLUZIONE CORRETTA:
5 long contatoreGiusto = lista.stream().count();
```

Da vecchi esami

CrittoCruciverba

```
1 public String parolaIniziale() {
2     List<String> parole;
3     Cruciverba cruciverba = this.getCruciverba(); // Ottieni l'oggetto Cruciverba dal Controller
4     parole = IntStream.range(0, cruciverba.getNumRighe()) // 1. Crea uno stream di indici: da 0 a (numRighe - 1)
5         .mapToObj(cruciverba::paroleInRiga) // 2. Per ogni indice, ottieni l'array String[] delle parole
6         .flatMap(Arrays::stream) // 3. Appiattisci tutti gli String[] in un unico Stream<String>
7         .filter(p -> this.numeroCaratteriDistinti(p) == this.getModalita().numeroCaratteriDistinti()) // filtro per lunghezza
8         .collect(Collectors.toList());
9
10    int i = r.nextInt(parole.size());
11    return parole.get(i);
12 }
```